Série d’exercices — Langage C

Avec corrections complètes

Contents

Partie A — Variables, types, opérateurs 1

Partie B — Conditions et boucles 3

Partie C — Tableaux et chaînes 5

Partie D — Fonctions et pointeurs 7

Partie E — Structures et allocation dynamique 9

Partie F — Fichiers 11

Partie G — Préprocesseur et macros 12

Partie A — Variables, types, opérateurs

Exercice 1. Déclarez un `int`, un `float` et un `char`, affichez-les avec `printf`.

```c
// Correction
#include <stdio.h>
int main(void) {
    int a = 10;
    float b = 3.5f;
    char c = 'K';
    printf("%d %.1f %c\n", a, b, c);
    return 0;
}
```

Exercice 2. Écrivez un programme qui échange deux entiers `a` et `b` sans variable temporaire (astuce arithmétique).

```c
// Correction
#include <stdio.h>
int main(void) {
    int a = 5, b = 8;
    a = a + b;
    b = a - b;
    a = a - b;
    printf("a=%d b=%d\n", a, b); // a=8 b=5
    return 0;
}
```

Exercice 3. Testez la différence entre `7/2` et `7.0/2`.

```c
// Correction
#include <stdio.h>
int main(void) {
    printf("%d\n", 7/2);     // 3
    printf("%.1f\n", 7.0/2); // 3.5
    return 0;
}
```

Exercice 4. Écrivez un programme qui calcule le reste et le quotient de la division de deux entiers saisis par l'utilisateur.

```c
// Correction
#include <stdio.h>
int main(void) {
    int a, b;
    printf("Entrez deux entiers : ");
    scanf("%d %d", &a, &b);
    printf("Quotient: %d, Reste: %d\n", a/b, a%b);
    return 0;
}
```

Partie B — Conditions et boucles

Exercice 5. Écrivez un programme qui détermine si un nombre est pair ou impair.

```c
// Correction
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);
    if (n % 2 == 0) printf("Pair\n");
    else printf("Impair\n");
    return 0;
}
```

Exercice 6. Écrivez un programme qui affiche la table de multiplication d'un nombre saisi (de 1 à 10).

```c
// Correction
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);
    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n*i);
    }
    return 0;
}
```

Exercice 7. Calculez la somme des entiers de 1 à N avec une boucle `while`.

```c
// Correction
#include <stdio.h>
int main(void) {
    int n, somme = 0, i = 1;
    scanf("%d", &n);
    while (i <= n) { somme += i; i++; }
    printf("Somme = %d\n", somme);
    return 0;
}
```

Exercice 8. À l'aide d'un `switch`, affichez le nom du mois correspondant à un numéro (1-12).

```c
// Correction
#include <stdio.h>
int main(void) {
    int mois;
    scanf("%d", &mois);
    switch (mois) {
        case 1: printf("Janvier\n"); break;
        case 2: printf("Février\n"); break;
        case 3: printf("Mars\n"); break;
        default: printf("Autre mois\n");
    }
    return 0;
}
```

Exercice 9. Vérifiez si un nombre est premier.

```c
// Correction
#include <stdio.h>
int main(void) {
    int n, premier = 1;
    scanf("%d", &n);
    if (n < 2) premier = 0;
    for (int i = 2; i*i <= n && premier; i++) {
        if (n % i == 0) premier = 0;
    }
    printf(premier ? "Premier\n" : "Non premier\n");
    return 0;
}
```

Partie C — Tableaux et chaînes

Exercice 10. Remplissez un tableau de 10 entiers saisis au clavier, puis affichez leur somme et leur moyenne.

```c
// Correction
#include <stdio.h>
int main(void) {
    int t[10], somme = 0;
    for (int i = 0; i < 10; i++) scanf("%d", &t[i]);
    for (int i = 0; i < 10; i++) somme += t[i];
    printf("Somme=%d Moyenne=%.2f\n", somme, somme/10.0);
    return 0;
}
```

Exercice 11. Trouvez le maximum et le minimum d'un tableau d'entiers.

```c
// Correction
#include <stdio.h>
int main(void) {
    int t[5] = {12, 45, 3, 89, 22};
    int max = t[0], min = t[0];
    for (int i = 1; i < 5; i++) {
        if (t[i] > max) max = t[i];
        if (t[i] < min) min = t[i];
    }
    printf("Max=%d Min=%d\n", max, min);
    return 0;
}
```

Exercice 12. Inversez un tableau d'entiers sur place.

```c
// Correction
#include <stdio.h>
int main(void) {
    int t[5] = {1,2,3,4,5};
    for (int i = 0; i < 5/2; i++) {
        int tmp = t[i];
        t[i] = t[4-i];
        t[4-i] = tmp;
    }
    for (int i = 0; i < 5; i++) printf("%d ", t[i]); // 5 4 3 2 1
    return 0;
}
```

Exercice 13. Écrivez une fonction qui compte le nombre de voyelles dans une chaîne de caractères.

```c
// Correction
#include <stdio.h>
#include <string.h>

int compterVoyelles(char *s) {
    int compte = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c=='a'||c=='e'||c=='i'||c=='o'||c=='u'||
```

```
            c=='A'||c=='E'||c=='I'||c=='O'||c=='U') {
            compte++;
        }
    }
    return compte;
}

int main(void) {
    char phrase[100] = "Bonjour tout le monde";
    printf("Voyelles: %d\n", compterVoyelles(phrase));
    return 0;
}
```

Exercice 14. Vérifiez si une chaîne est un palindrome (sans utiliser de bibliothèque externe pour l'inversion).

```
// Correction
#include <stdio.h>
#include <string.h>

int estPalindrome(char *s) {
    int len = strlen(s);
    for (int i = 0; i < len/2; i++) {
        if (s[i] != s[len-1-i]) return 0;
    }
    return 1;
}

int main(void) {
    char mot[50] = "radar";
    printf(estPalindrome(mot) ? "Palindrome\n" : "Pas palindrome\n");
    return 0;
}
```

Partie D — Fonctions et pointeurs

Exercice 15. Écrivez une fonction `int max3(int a, int b, int c)` qui renvoie le plus grand des trois.

```c
// Correction
#include <stdio.h>
int max3(int a, int b, int c) {
    int m = a;
    if (b > m) m = b;
    if (c > m) m = c;
    return m;
}
int main(void) {
    printf("%d\n", max3(5, 12, 9)); // 12
    return 0;
}
```

Exercice 16. Écrivez une fonction qui échange deux entiers via des pointeurs.

```c
// Correction
#include <stdio.h>
void echanger(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
int main(void) {
    int x = 3, y = 9;
    echanger(&x, &y);
    printf("x=%d y=%d\n", x, y); // x=9 y=3
    return 0;
}
```

Exercice 17. Écrivez une fonction récursive qui calcule la somme des chiffres d'un entier.

```c
// Correction
#include <stdio.h>
int sommeChiffres(int n) {
    if (n == 0) return 0;
    return (n % 10) + sommeChiffres(n / 10);
}
int main(void) {
    printf("%d\n", sommeChiffres(1234)); // 10
    return 0;
}
```

Exercice 18. Écrivez une fonction qui prend un tableau et sa taille et qui trie ce tableau (tri à bulles).

```c
// Correction
#include <stdio.h>
void triBulles(int t[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-1-i; j++) {
            if (t[j] > t[j+1]) {
                int tmp = t[j];
```

```
                t[j] = t[j+1];
                t[j+1] = tmp;
            }
        }
    }
}
int main(void) {
    int t[5] = {5,2,9,1,7};
    triBulles(t, 5);
    for (int i = 0; i < 5; i++) printf("%d ", t[i]); // 1 2 5 7 9
    return 0;
}
```

Partie E — Structures et allocation dynamique

Exercice 19. Définissez une structure `Point` (x, y) et une fonction qui calcule la distance entre deux points.

```c
// Correction
#include <stdio.h>
#include <math.h>

typedef struct { double x, y; } Point;

double distance(Point a, Point b) {
    return sqrt((a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y));
}

int main(void) {
    Point p1 = {0, 0}, p2 = {3, 4};
    printf("%.2f\n", distance(p1, p2)); // 5.00
    return 0;
}
```

Exercice 20. Créez un tableau dynamique de N entiers saisi par l'utilisateur (N donné au clavier), remplissez-le, affichez-le, puis libérez la mémoire.

```c
// Correction
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n;
    printf("Combien d'éléments ? ");
    scanf("%d", &n);

    int *t = malloc(n * sizeof(int));
    if (t == NULL) { return 1; }

    for (int i = 0; i < n; i++) scanf("%d", &t[i]);
    for (int i = 0; i < n; i++) printf("%d ", t[i]);

    free(t);
    return 0;
}
```

Exercice 21. Créez une structure `Etudiant` (nom, note), un tableau dynamique de 3 étudiants, et affichez celui qui a la meilleure note.

```c
// Correction
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char nom[30];
    float note;
} Etudiant;
```

```c
int main(void) {
    Etudiant *t = malloc(3 * sizeof(Etudiant));
    strcpy(t[0].nom, "Awa");   t[0].note = 14.5;
    strcpy(t[1].nom, "Ibra");  t[1].note = 17.0;
    strcpy(t[2].nom, "Sokhna"); t[2].note = 12.0;

    int meilleur = 0;
    for (int i = 1; i < 3; i++) {
        if (t[i].note > t[meilleur].note) meilleur = i;
    }
    printf("Meilleur: %s (%.1f)\n", t[meilleur].nom, t[meilleur].note);

    free(t);
    return 0;
}
```

Partie F — Fichiers

Exercice 22. Écrivez un programme qui enregistre 5 nombres saisis dans un fichier `notes.txt`, puis relit et affiche leur somme.

```c
// Correction
#include <stdio.h>

int main(void) {
    FILE *f = fopen("notes.txt", "w");
    if (f == NULL) return 1;
    for (int i = 0; i < 5; i++) {
        int n;
        scanf("%d", &n);
        fprintf(f, "%d\n", n);
    }
    fclose(f);

    FILE *f2 = fopen("notes.txt", "r");
    int val, somme = 0;
    while (fscanf(f2, "%d", &val) == 1) {
        somme += val;
    }
    fclose(f2);
    printf("Somme = %d\n", somme);
    return 0;
}
```

Partie G — Préprocesseur et macros

Exercice 23. Écrivez une macro `MAX(a,b)` bien parenthésée et testez-la avec des expressions (ex : `MAX(2+3, 4)`).

```c
// Correction
#include <stdio.h>
#define MAX(a,b) (((a) > (b)) ? (a) : (b))

int main(void) {
    printf("%d\n", MAX(2+3, 4)); // 5, correct grâce aux parenthèses
    return 0;
}
```